

Experimental Project

Piter Garcia and Dinesh Dommaraju

ABSTRACT

The goal of the project is to observe empirically the complexities and time measuring routines of different implementations of algorithms. We aim to find the Longest Common Subsequence from the assigned algorithms after feeding these with two strings. In particular, they will be tested in the scenarios where the length of the strings will very randomly and linearly.

1. OVERVIEW

In this project we are going to analyzed four different algorithms to find the Longest Common Subsequence. Each one of these algorithms will be analyzed in terms of its source code, time measuring routines, sample profile, and high level script(s). First, in the source code section, we will describe the implementation of the recursive function. Second, in the time measuring routines, we will analyze the result or LCS length in terms of the number of recursive calls that the algorithm executed. The time taken by each algorithm to execute the recursive calls, or one test case, will recorded and analyzed by high level scripts with respect to the algorithm's parameters. Third, in the sample profile run section, we will give a demonstration of how the algorithm's behavior when executing the large number of test cases. Lastly, in the high level scripts, we will document the implementation of each algorithm to analyze its performance. This involves managing IO files, random input generation, and graphing. The objective is to obtain the necessary documentation to analyze all algorithms as accurate as possible.

1.1 Naive Recursive

Naive Recursive calculates the LCS in Exponential time. The complexity is $O(2^n)$ in the worst case. The worst case happens when both the strings mismatch and their LCS is 0. The naïve recursive approach is not efficient in time as it computes the same sub-problem repeated number of times. The behavior of the Naive recursive is shown in the figures 2, 3, and 4 of this project.

1.4 Quadratic-time linear-space Version

Naive Recursive:
Source code:

```
public class NaiveRecursive
{
    String str1, str2;

    public int LCS(int xLen, int yLen)
    {
        this.recursiveCalls++;
        if (xLen==1||yLen==1)
            return 0;
        else if(this.str1.charAt(xLen)==this.str2.charAt(yLen))
            return 1+LCS(xLen-1,yLen-1);
        else
            return max(LCS(xLen-1,yLen),LCS(xLen,yLen-1));
    }

    public int max(int a, int b)
    {
        return (a > b)?a : b;
    }
}
```

Figure 1: Source Code: Naive Recursive.

The source code for the Naive Recursive algorithm can be seen in the figure 1.

Time Measuring Routines and Analysis:

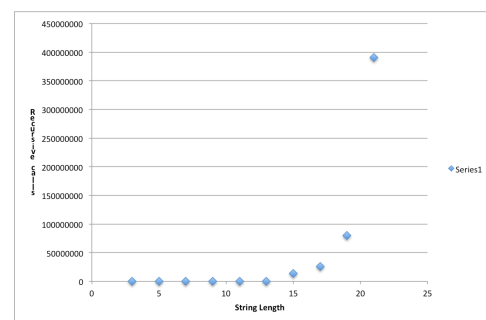


Figure 2: Recursive Calls vs String Length.

We can observe that the number of recursive calls increases exponentially as the input strings length increases.

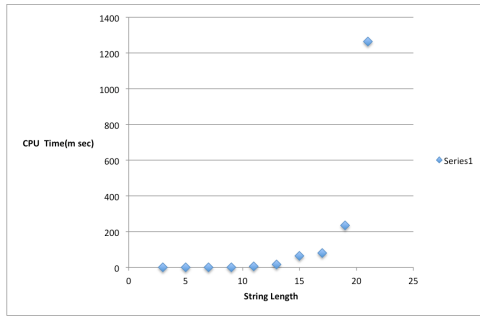


Figure 3: CPU-Time vs String Length.

String Length	LCS Length	CPU Time (msecs)	Number of recursive calls
4	2	0	63
9	6	8	11560
17	8	80	28288708
23	13	12732	

Figure 4: Sample Output: Naive Recursive.

The Algorithm takes more than 10 sec when the input strings are of length around 20 and it runs for longer time even if the length is around 30.

The naive recursive is inefficient in terms of time taken but it is easier to implement and it doesn't take extra space.

1.2 Memoization Version

```
public int LCSm(int i, int j){
    recursiveCalls++; //keeps track of calls to printLCS

    if(i == -1 || j == -1) {
        return 0;
    }else if(c[i][j] == -1) {
        if (str1.charAt(i) == str2.charAt(j)){
            c[i][j] = 1 + LCSm(i-1,j-1);
        }
        else {
            c[i][j] = Math.max(LCSm(i-1,j), LCSm(i,j-1));
        }
    }
    return c[i][j];
}
```

Figure 5: Source Code1: Recursive Algorithm

The source code of the Memoization Algorithm version is shown in the figures 4 and 5 of this project.

```
public void Memoization(String x, String y) {
    str1 = x;
    str2 = y;
    c = new int[str1.length()][str2.length()];

    for(int i=0; i<str1.length(); i++) {
        for(int j=0; j<str2.length(); j++) {
            c[i][j] = -1;
        }
    }
    recursiveCalls = 0;
}
```

Figure 6: Source Code2: Memoization

The figure 5 is a method that belongs the source code of the Memoization Algorithm version.

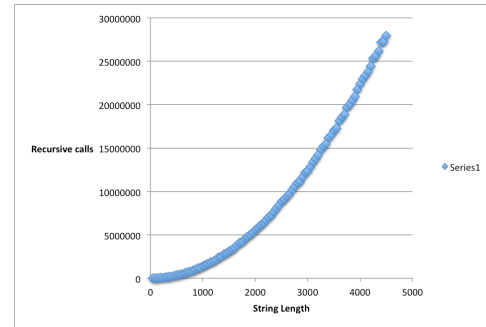


Figure 7: RecursiveCalls vs StringLength: Memoization

Memoization finds the longest common subsequence similar to Naive Recursive Algorithm but instead of a recursive call as in the case of naive recursive every time it computes the whether there is a entry in the table (Previous stored values). If its there it takes the result from these tables else it performs a recursive call and stored the result in the table.

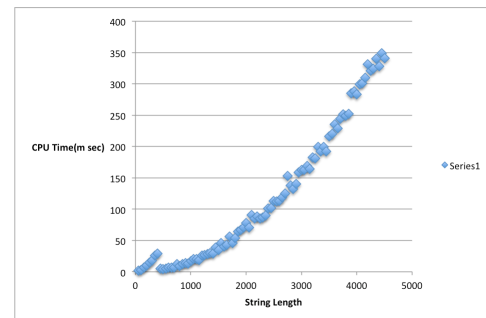


Figure 8: CPU TIME vs String Length: Memoization

The time complexity of Longest Common Subsequence is $O(n^3)$. As we are storing the intermediate results in a table which requires extra space. As the string size increases the table size increases (As in the Analysis below, when the string length is greater than 5690) and it run out of space throwing "Out of Memory Error: Java Heap Space".

String Length	LCS Length	CPU Time(msecs)	Number of recursive calls	Table size
32	18	1	57	
64	36	2	6222	
1024	620	63	1446446	
4096	2474	307	23366199	
> 5690	Out of Memory	java Heap Space		

Figure 9: Output Sample: Memoization

The behavior of the algorithm is shown in the figures 7, 8 and 9.

Thus, Memoization is efficient when the strings are of smaller length and is not good to use when the strings are of larger length because of memory usage (As discussed above) and cubical time growing.

Its not possible to reach scenario where the algorithm takes more than 10 secs since It runs out of space.

1.3 Dynamic programming Version

```

public String LCSd(String X, int i, int j) {
    recursiveCalls++; //keeps track of calls to printLCS
    String retVal = "";

    if(i < 0 || j < 0) {
        retVal = "" + retVal;
    } else {
        if(b[i][j] == DIAGONAL) {
            retVal += LCSd(X, i-1, j-1);
            retVal += str1.charAt(i);

        } else if(b[i][j] == UP) {
            retVal += LCSd(X, i-1, j);

        } else {
            retVal += LCSd(X, i, j-1);
        }
    }

    return retVal;
} //printLCS

```

Figure 10: Source Code1: Dynamic Algorithm

The figure 10 is the source code for the Dynamic Algorithm, and the figure 11 is a method that belongs to the source code of the Dynamic Algorithm.

```

public void lcsLength(String x, String y) {
    int m = x.length();
    int n = y.length();
    b = new char[m][n];
    c = new int[m+1][n+1];
    // Initialize the c table.
    for(int i=1; i<=m; i++) {
        c[i][0] = 0;
        for(int j=0; j<=n; j++) {
            c[0][j] = 0;
        }
    }
    // Construct the c and b tables.
    for(int i=0; i<=m; i++) {
        for(int j=0; j<=n; j++) {
            if(x.charAt(i) == y.charAt(j)) {
                c[i+1][j+1] = c[i][j]+1;
                b[i][j] = DIAGONAL;
            } else if(c[i][j+1] >= c[i+1][j]) {
                c[i+1][j+1] = c[i][j+1];
                b[i][j] = UP;
            } else {
                c[i+1][j+1] = c[i+1][j];
                b[i][j] = LEFT;
            }
        }
    }
} //lcsLength

```

Figure 11: Source Code2: Dynamic Algorithm

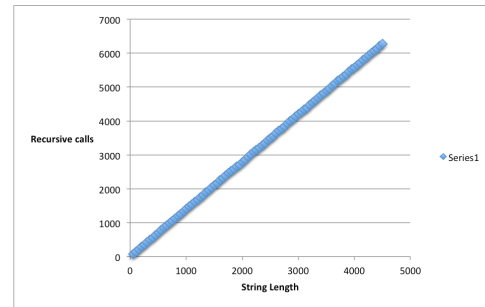


Figure 12: Recursive Calls vs String: Dynamic Programming
As Contrary to Memoization, Dynamic Programming follows the Bottom up approach. It solves the sub problems first, filling up an n-dimensional table. Based on the results in the table, the solution to the top original problem is then computed. The time complexity of the LCS using Dynamic Programming is $O(mn)$, where m and n are the length of the input strings. Thus Dynamic programming solves the LCS in Quadratic time but it also takes quadratic space. The performance of the Dynamic Programming algorithm can be observed in the figures 12, 13, and 14 in this project.

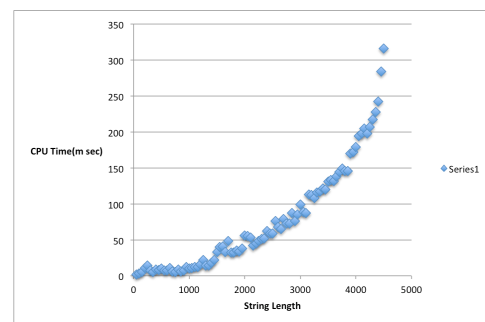


Figure 13: CPU Time vs String Length: Dynamic Programming

As it also takes Quadratic space, It suffers from the limitation which we have faced for Memoization. It cannot take handle strings of length greater than 5690 as the program runs out of memory throwing exception Out of Memory Error: Java Heap Space. But it is efficient as compared to Memoization as it runs it quadratic time contrary to Memoization, which takes cubic time.

String Length	LCS Length	CPU Time(m secs)	Number of recursive calls	Table size
32	18	1	47	
64	34	1	93	
1024	611	15	1437	
4096	2472	252	5720	
> 5690	Out of Memory: java Heap Space			

Figure 14: Sample Output:

Its not possible to reach scenario where the algorithm takes more than 10 sec since It runs out of space similar to Memoization.

```

public String startHirschberg(String string1, String string2, int stringlen, int stringlen2)
{
    this.recursiveCalls++;
    String temp="";
    if(stringlen==0 || stringlen2==0)
    {
        temp="";
    }
    else if(stringlen==2)
    {
        temp="";
        for(int i=0;i<stringlen2;i++)
        {
            if(string1.charAt(i)==string2.charAt(i))
            {
                temp+=string1.charAt(i);
            }
        }
    }
    else
    {
        int mid=stringlen/2;
        int[] ScoreL= HNScore(mid, stringlen, string1.substring(0, mid), string2);
        int[] ScoreR= HNScore(stringlen-mid, stringlen, reversedString(string1.substring(mid), reversedString(string2)));
        int ymid=PartitionY(ScoreL, ScoreR, stringlen);
        String tempL=startHirschberg(string1.substring(0, mid), string2.substring(0, ymid), mid, ymid);
        String tempR=startHirschberg(string1.substring(mid, string2), string2.substring(ymid, stringlen-mid), stringlen-mid, stringlen-ymid);
        temp=tempL+tempR;
    }
    return temp;
}
}

```

Figure 15: Source Code1: Hirschberg.

The source code for the Hirschberg algorithm is shown in the figure 15. Also, you can see two main methods that belong to the Hirschberg algorithm. These methods are shown in figures 16 and 17.

```

public int[] HNScore(int stringlen, int string2len, String string1, String string2 )
{
    int[][] Score=new int[2][string2len+1];
    for(int i=0;i<string2len;i++)
    {
        Score[i][1]=0;
    }
    for(int i=1;i<stringlen;i++)
    {
        for(int j=0;j<string2len;j++)
        {
            Score[i][j]=Score[i][j-1];
        }
        for(int j=1;j<string2len;j++)
        {
            if(string1.charAt(i-1)==string2.charAt(j-1))
            {
                Score[i][j]=Score[i-1][j-1]+1;
            }
            else
            {
                Score[i][j]=max (Score[i][j-1], Score[i-1][j]);
            }
        }
    }
    return Score[i];
}
}

```

Figure 16: Source Code2: Hirschberg.

```

public int PartitionY(int [] ScoreL, int[] ScoreR, int n)
{
    int m=0, k=0;
    for(int j=0;j<=n;j++)
    {
        if(m<(ScoreL[j]+ScoreR[n-j]))
        {
            m = ScoreL[j]+ScoreR[n-j];
            k=j;
        }
    }
    return k;
}
}

```

Figure 17: Source Code3: Hirschberg.

Hirschberg finds the longest Common subsequence by recursively dividing the problem into sub-problem, It finds the LCS in a quadratic time and in linear space, An improvement over Dynamic Programming Algorithm which takes quadratic time and quadratic space. The algorithm takes $O(p(m+1-p)\log n)$ where p is the length of the Longest common subsequence. m and n are lengths of the Input strings such that $m \leq n$. The behavior of the algorithm is shown in the figures 18,19, and 20 in this project.

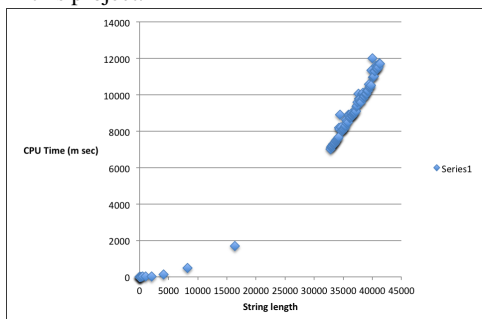


Figure 18: CPU-Time vs String Length: Hirschberg.

The Algorithm takes more than 10 sec when the input strings are of length around 20 and it runs for longer time even if the length is around 30.

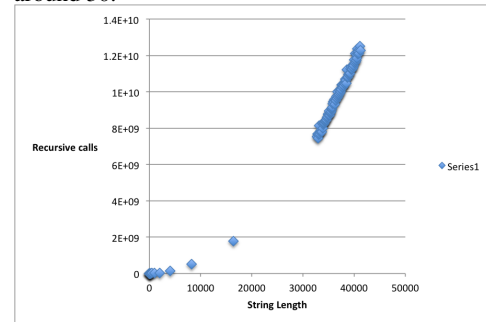


Figure 19: Recursive Calls vs String Length: Hirschberg.

We can observe that the number of recursive calls increases exponentially as the input strings length increases.

String Length	LCS Length	CPU Time(m secs)	Number of recursive calls
1024	611	62	1851
4096	2470	175	7455
16384	9919	1794	29687
32768	19905	8375	59605
34000	20145	11478	74795

Figure 20: Sample Output: Hirschberg.

The Algorithm takes more than 10 sec when the Input size is approximately greater than the 34000. The Algorithm is efficient as it can process longer string in very less time than as compared to other algorithms discussed above in the previous sections.

The naive recursive is inefficient in terms of time taken but it is easier to implement and it doesn't take extra space.

Q-RC	Q-CT (ms)	LCS	ISL1	ISL2	Q-RC	Q-CT (ms)	LCS	ISL1	ISL2
3	0	0	2	2	66425	8973	22037	36368	36368
5	0	1	4	4	66727	8809	22133	36468	36468
13	0	3	8	8	66809	8836	22175	36568	36568
27	0	7	16	16	67101	8933	22277	36668	36668
53	1	17	32	32	67367	8962	22292	36768	36768
111	3	37	64	64	67427	9013	22342	36868	36868
227	7	74	128	128	67659	9015	22407	36968	36968
457	7	150	256	256	67947	9152	22503	37068	37068
929	14	304	512	512	67995	9094	22552	37168	37168
1817	14	609	1024	1024	68219	9153	22648	37268	37268
3735	34	1243	2048	2048	68351	9410	22644	37368	37368
7439	144	2475	4096	4096	68403	9602	22694	37468	37468
14907	484	4964	8192	8192	68819	10042	22763	37568	37568
29997	1704	9943	16384	16384	68983	9781	22836	37668	37668
59805	7017	19906	32768	32768	69119	9758	22908	37768	37768
59879	7124	19926	32868	32868	69455	9646	22954	37868	37868
60035	7148	19964	32968	32968	69445	9581	23040	37968	37968
60263	7204	20064	33068	33068	69593	9598	23053	38068	38068
60441	7199	20120	33168	33168	70077	10014	23159	38168	38168
60667	7292	20186	33268	33268	70379	10043	23211	38268	38268
60719	7288	20232	33368	33368	70427	10122	23280	38368	38368
61095	7339	20332	33468	33468	70439	9799	23321	38468	38468
61145	7382	20335	33568	33568	70835	9928	23381	38568	38568
61347	7457	20440	33668	33668	70793	9913	23455	38668	38668
61641	7482	20528	33768	33768	71063	10039	23529	38768	38768
61729	7525	20523	33868	33868	71171	10053	23609	38868	38868
62013	7554	20632	33968	33968	71457	10055	23607	38968	38968
62175	7629	20633	34068	34068	71597	10220	23675	39068	39068
62329	7680	20697	34168	34168	71685	10207	23767	39168	39168
62551	8187	20781	34268	34268	71995	10215	23798	39268	39268
62649	8105	20863	34368	34368	72259	10582	23840	39368	39368
62855	8909	20897	34468	34468	72365	10383	23942	39468	39468
63317	8227	20966	34568	34568	72411	10452	23981	39568	39568
63263	8019	21027	34668	34668	72763	10449	24090	39668	39668
63659	8002	21103	34768	34768	72941	10534	24103	39768	39768
63757	8056	21205	34868	34868	73155	11346	24187	39868	39868
63927	8104	21206	34968	34968	73247	12012	24248	39968	39968
64077	8113	21296	35068	35068	73259	10965	24285	40068	40068
64407	8163	21323	35168	35168	73785	10996	24409	40168	40168
64299	8233	21432	35268	35268	73965	11023	24437	40268	40268
64613	8299	21433	35368	35368	74171	11306	24509	40368	40368
64829	8620	21519	35468	35468	74141	11287	24545	40468	40468
64963	8436	21626	35568	35568	74499	11578	24608	40568	40568
65231	8553	21675	35668	35668	74787	11530	24680	40668	40668
65333	8481	21683	35768	35768	74963	11385	24744	40768	40768
65413	8916	21729	35868	35868	75299	11427	24799	40868	40868

Figure 21: Sample Profile Run: Hirschberg.

The sample profile run can be seen in the figure 21 and 22.

65697	8865	21794	35968	35968	75019	11559	24826	40968	40968
65873	8797	21885	36068	36068	75601	11514	24937	41068	41068
66151	8701	21958	36168	36168	75809	11719	24952	41168	41168
66397	8770	21982	36268	36268	75869	11688	25018	41268	41268
Total Time:801193mSec									
Total Time Elapsed:801377mSec									
Total Time Elapsed:80135c									

Figure 22: Sample Profile Run: Hirschberg.

Analyzing the behavior of all the four algorithms discussed above using the same Input cases. We get the results shown in the figures 23, 24, 25 and 26.

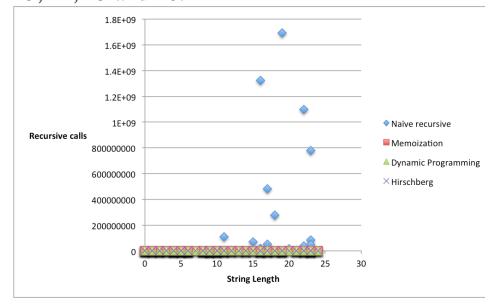


Figure 23: Recursive Calls vs String Length: All Algorithms

As we can observe, the number of recursive calls for algorithms other than naive recursive is negligible compared to naive recursive algorithm since the naive recursive takes exponential time to find the LCS.

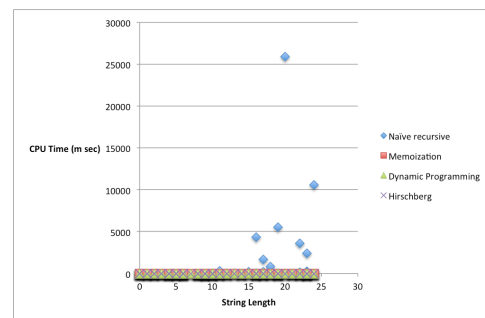


Figure 24: CPU TIME vs String Length: All Algorithms

Same as for recursive calls, the CPU time for algorithms other than naive recursive is negligible compared to naive recursive algorithm since the naive recursive takes exponential time to find the LCS.

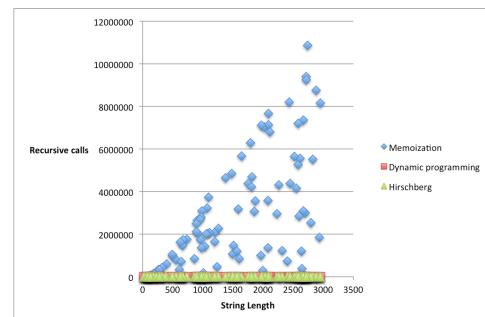


Figure 25: CPU TIME vs String Length: All Algorithms Except Naive Recursive]

We have excluded the naive recursive approach to get an effective comparison of the Memoization, Dynamic Programming and Hirschberg algorithms. We can observe that the Memoization has more number of recursive calls than the Dynamic Programming and Hirschberg, which has cubic time complexity as compared to the Hirschberg and Dynamic Programming algorithms. It also indicates that the dynamic programming and Hirschberg algorithms have similar number of recursive calls for same input string.

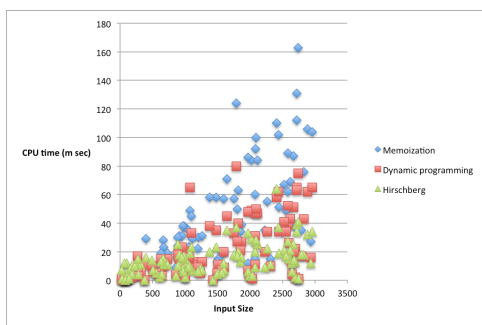


Figure 26: CPU TIME vs InputSize: For all algorithms except Naive Recursive algorithms.

Though, the Hirschberg and Dynamic Programming has similar number of recursive calls but Hirschberg outperforms the dynamic programming comparatively in CPU time taken to process the same Input random Strings. The figure of these measurements can be seen in the figure 26.

2. ALGORITHMS HIGH LEVEL SCRIPT

```

dirList = dir();
names = (dirList.name);
outNames = {}; data = {};
A = {}; B = {}; C = {}; D = {}; E = {};
% graph one
c=0; p = 0;
for i=1:numel(names)
    name = names(i);
    if ~isequal(name, '.') &&
    ~isequal(name, '..')
        [~,name] = fileparts(name(i));
        outNames(end+1) = name;
        if strcmp(name, names(i)) == 0
            % open the file and read it
            file = importdata(name(i));
            disp(A);
            rCN = sort(file.data(:,1));
            cpuT = sort(file.data(:,2));
            lcsL = sort(file.data(:,3));
            isL1 = sort(file.data(:,4));
            isL2 = sort(file.data(:,5));
            x = cpuT; y1 = rCN; y2 = lcsL;
            z = isL1;
            A(end+1) = rCN; B(end+1) = cpuT; C(end+1) = lcsL;
            D(end+1) = isL1; E(end+1) = isL2;
            % graph one
            figure;
            subplot(2,2,1)
            figure;
            cc=hsb(12);
            subplot(3,3,1)
            hold on;
            for i=1:4
                plot(A(i), 'color', cc(i,:));
            end
            legend('DB SIZE', 0);
            title('Recursive Calls Comparison');
            xlabel('Double');
            ylabel('Size');
            subplot(3,3,2)
            cc=hsb(12);
            hold on;
            for i=1:4
                plot(B(i), 'color', cc(i,:));
            end
            legend('DB RSIZE', 0);
            title('CPU Time Comparison');
            xlabel('Double');
            ylabel('Size');
            subplot(3,3,3)
            cc=hsb(12);
            hold on;
            for i=1:4
                plot(C(i), 'color', cc(i,:));
            end
            legend('DB RSIZE', 0);
            title('SECOND INPUT COMPARISON');
            xlabel('Double');
            ylabel('Size');
        end
        plot(x, y1, 'r*');
        legend('recursive calls No. AND x= CPU-Time.', 0);
        title('CPU-Time vs Recursive Calls Numbers');
        xlabel('CPU-TIME (ms)');
        ylabel('RECURSIVE CALLS NO. ');
        % graph two
        subplot(2,2,2)
        plot(x, y2, 'r*'); % lines only!
        legend('LCS-Length AND x= CPU-Time.', 0);
        title('CPU-Time vs LCS-Length');
        xlabel('CPU-TIME (ms)');
        ylabel('LCS-LENGTH');
        % graph two
        subplot(2,2,3)
        plot(x, y, 'r*'); % lines only!
        legend('ISL1-Length AND x= CPU-Time.', 0);
        title('CPU-Time vs ISL1-Length');
        xlabel('CPU-TIME (ms)');
        ylabel('ISL1-LENGTH');
        % graph two
        subplot(2,2,4)
        plot(x, z, 'r*'); % lines only!
        legend('ISL2-Length AND x= CPU-Time.', 0);
        title('CPU-Time vs ISL2-Length');
        xlabel('CPU-TIME (ms)');
        ylabel('ISL2-LENGTH');
    end
end
end
plot(C(i), 'color', cc(i,:));
end
legend('DB SIZE', 0);
title('LCS SIZE COMPARISON');
xlabel('Double');
ylabel('Size');
subplot(3,3,4)
cc=hsb(12);
hold on;
for i=1:4
    plot(D(i), 'color', cc(i,:));
end
legend('DB RSIZE', 0);
title('FIRST INPUT COMPARISON');
xlabel('Double');
ylabel('Size');
subplot(3,3,5)
cc=hsb(12);
hold on;
for i=1:4
    plot(E(i), 'color', cc(i,:));
end
legend('DB RSIZE', 0);
title('SECOND INPUT COMPARISON');
xlabel('Double');
ylabel('Size');

```

Figure 27: Matlab - High Level Script: To Test All Algorithm
 The code in the figure 27 is the high level script we use to analyze the performance and behavior of each one of the algorithm already tested and individually analyzed above. This script takes the time sequence, number of recursive calls, and LCS length along with its inputs lengths recorder by other high level scripts and graphs their performance for comparison purpose. In fact, this high level script takes the components mentioned above of each algorithm and graph them separately for a better understanding of these algorithm performance.

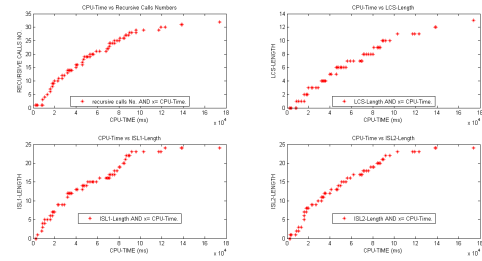


Figure 28: Matlab - High Level Script: To Test Dynamic Programming
 The performance of the dynamic programming algorithm is shown in the figure 28.

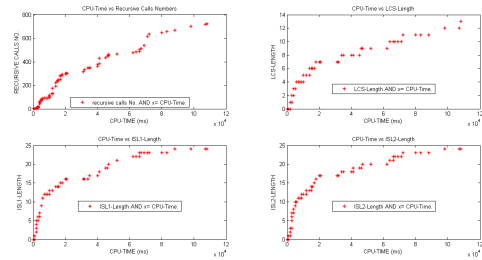


Figure 29: Matlab - High Level Script: To Test Memoization
 Also, the figure for the Memoization version of the recursive algorithm can be seen in the figure 29.

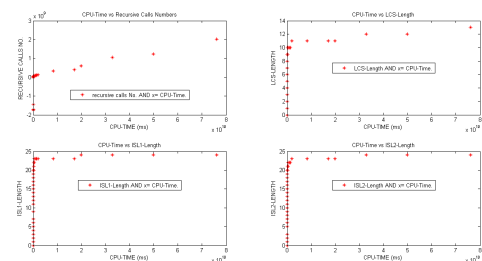


Figure 30: Matlab - High Level Script: To Test Naive Recursive Algorithm
 The performance of Naive Recursive algorithm can be seen in figure 30.

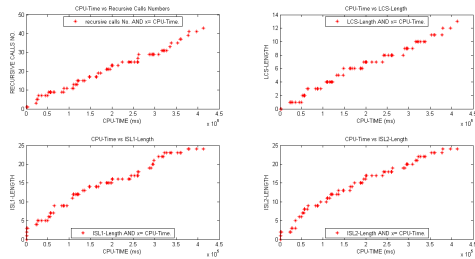


Figure 31: Matlab - High Level Script: To Test Hirschberg Algorithm

Similarly, we can see the performance of the Hirschberg algorithm in the figure 31.

```
public class TestAlgorithms {
    private int choice = 0;
    static String[] list = {"NaiveRecursive.txt",
        "DynamicAV.txt", "MemoizationAV.txt",
        "QuadraticAV.txt"};
    static String
        path = "Users/dineshdharmaraju/Documents/workspac
        e/Algorithms/src/";

    public void getCh(int x){
        choices = x;
        System.out.println(x);
    }

    public static void main(String[] args) throws
        Exception{
        Scanner in = new Scanner(System.in);
        TestAlgorithms test = new TestAlgorithms();

        System.out.println("Please Enter the Algorithm
        You Wish to Run:");
        int c=in.nextInt();
        if(c==4)
            testAll();
        else
        {
            test.setCh(c);
            try
            {
                test.test();
            } catch (FileNotFoundException e) {
                e.printStackTrace();
            }
        }

        public static void testAll() throws Exception
        {
            File file[] = new File[4];
            FileWriter fw[] = new FileWriter[4];
            BufferedWriter bw[] = new BufferedWriter[4];
            long t1, t2;
            int NumberOfTestcases=80;
            GenerateInputs nrl = new GenerateInputs();

            for (int i=0;i<4;i++){
                file[i]=new File(path+list[i]);
                fw[i]= new
                    FileWriter(file[i].getAbsolutePath());
                bw[i] = new BufferedWriter(fw[i]);
                bw[i].write("Recursive Calls" + " " + "CPU TIME
                    (ms)" + " " + "LCS" + " " + "ISL1" + " " + "ISL2\n");
                LCS lcs;
            }

            for(int i=0; i<NumberOfTestcases; i++){
                System.out.println(" "+i);
                nrl.GenerateTestcases(i);

                for (int j=1;j<4;j++){
                    t1 = System.nanoTime();
                    System.out.println(" "+t1);
                    lcs = new LCS(nrl.getStr1(), nrl.getStr2(), j);
                    t2 = System.nanoTime();
                    bw[j].write(lcs.getRC() +
                        "\t" + (t2-t1) + "\t" + lcs.getLCS() +
                        "\t" + nrl.getStr1() + "\t" + " " +
                        nrl.getStr2() + "\n");
                }

                for(int i=0;i<4;i++){
                    bw[i].close();
                    System.out.println("Done");
                }

                public void test() throws FileNotFoundException
                {
                    long starttime = System.currentTimeMillis();
                    int NumberOfTestcases=100;
                    long totaltime=0;

                    GenerateInputs nrl = new GenerateInputs();
                    File file = null;

                    if(choice == 0)
                        file = new File(path+list[choice]);
                    else if( choice == 1)
                        file = new File(path+list[choice]);
                    else if(choice == 2)
                        file = new File(path+list[choice]);
                    else if(choice == 3)
                        file = new File(path+list[choice]);
                    else
                        file = new
                            File("Users/dineshdharmaraju/Documents/workspac
                            e/Algorithms/src/Test.txt");

                    // if file doesn't exists, then create it
                    if (!file.exists()) {
                        try {
                            file.createNewFile();
                        } catch (IOException e) {
                            e.printStackTrace();
                        }
                    }
                }
            }
        }
    }
}
```

Figure 33: Java Code - High Level Script: To Test All Algorithm

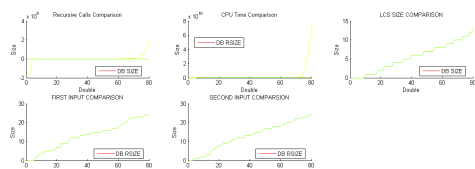


Figure 32: Matlab - High Level Script: To Test All Algorithm

In Matlab, we also were able to measure and compare the overall performance of all algorithms. This last test was performed by section, each graph represent a different section that contribute to the algorithm performance. These sections are the number of recursive calls, the cpu time, the LCS length and the length of the input strings.

```
FileWriter fw = null;
try {
    fw = new
        FileWriter(file.getAbsolutePath());
    } catch (IOException e2) {
        e2.printStackTrace();
    }

    BufferedWriter bw = new BufferedWriter(fw);
    try {
        bw.write("Recursive Calls" + " " + "CPU TIME
            (ms)" + " " + "LCS" + " " + "ISL1" + " " + "ISL2\n");
        e2.printStackTrace();
    } catch (IOException e2) {
        e2.printStackTrace();
    }

    for(int i=0; i<NumberOfTestcases; i++){
        try {
            nrl.GenerateTestcases(i);
        } catch (Exception e) {
            System.out.println("The Java Heap Source is Out
            of Memory");
            System.exit(0);
        }

        long t = 0;
        long t1 = 0;
        LCS lcs = null;
        try{
            t1 = System.nanoTime();
            lcs = new LCS(nrl.getStr1(), nrl.getStr2(),
                choice);

            } catch (Exception e) {
                e.printStackTrace();
            }

            if( choice < 4){
                t1 = System.nanoTime();
                lcs = new LCS(nrl.getStr1(), nrl.getStr2(),
                    choice);

                } else{
                    setCh(0);
                    test();

                    setCh(1);
                    test();

                    setCh(2);
                    test();

                    setCh(3);
                    test();

                    System.exit(1);
                } catch (Exception e) {
                    e.printStackTrace();
                }

                try {
                    bw.close();
                } catch (IOException e) {
                    e.printStackTrace();
                }

                System.out.println("Done");
            }

            System.out.println("Total Time Elapsed:");
            bw.write((endtime - starttime) + "mSec\n");
            bw.write("Total Time Elapsed:");
            bw.write((endtime - starttime)/1000 + "Sec\n");

            } catch (IOException e) {
                e.printStackTrace();
            }

            try {
                bw.close();
            } catch (IOException e) {
                e.printStackTrace();
            }

            System.out.println("Done");
        }
    }
}
```

Figure 34: Java Code - High Level Script: To Test All Algorithm

The figures 33 and 34 are the images of the source code we used to record the data of the time and recursive calls done by each one of the algorithm along with the length of its inputs and LCS. This code recored the time takes the time that each algorithm takes to generate the LCS from the two input strings. These data along with the number of recursive calls and length of the input and LCS string is recorded on a text file. These text files is later access by our Matlab and Excel High Level Script to simulate each algorithm performance.

3. SAMPLE PROFILES RUN

The three images in figure 35, 36 and 37 are a demonstration of the profile run of each one of the algorithms in nanoseconds.

M-RC	M-CT (ms)	D-RC	D-CT (ms)	Q-RC	Q-CT (ms)	LCS	ISL1	ISL2
476633	79	1699	49	1955	60	346	2139	346
7755178	107	3355	121	3897	35	1420	2059	2716
18636687	313	5348	179	7653	82	2222	4467	3102
14958320	17	1520	11	1647	10	614	866	1271
11817384	219	4106	112	5129	62	1768	2735	3141
16516793	218	5088	148	7283	64	2076	4343	2821
882879	10	2146	6	2599	3	463	920	714
12436015	147	4410	102	4619	59	1812	2384	3853
3310434	39	2225	22	3137	10	927	1842	1311
11663067	136	4188	80	5971	41	1740	3470	2464
1200901	16	2644	13	2983	10	536	3552	536
2879191	32	2707	22	3687	21	841	2611	937
10453188	121	4324	81	4051	49	1653	2079	3906
12023254	139	4283	118	4687	60	1765	2421	3628
193246	2	574	1	793	1	210	513	272
674677	8	2027	8	2359	7	424	3262	424
1315736	13	1377	6	1709	5	575	939	1013
2167947	2	1096	2	1121	2	193	1994	193
2077834	22	2095	15	2999	10	741	1962	873
21295113	278	5500	167	7523	88	2387	4193	3695
10333947	125	3848	86	5481	36	1636	3096	2390
433956	5	1470	4	1797	2	322	1489	324
371893	4	1412	5	617	6	309	309	1865
3612897	41	2362	26	3353	13	964	1989	1336
6655574	79	3907	51	5523	25	1288	3761	1455
12617509	150	4301	104	6149	53	1811	3528	2583
781520	8	1104	4	1151	3	440	595	952
6791565	80	3928	49	2953	35	1311	1487	3753
7600554	87	4500	58	2957	42	1379	1486	4414
24578288	306	5891	212	7327	94	2531	3957	4469
605556	7	1847	9	2169	7	384	3740	384
101003	2	817	0	295	2	148	148	931
749008	11	2176	10	811	21	406	406	3737
7661007	94	3525	56	5089	26	1404	3070	1862
22572444	275	5706	169	6913	87	2444	3709	4442
1468509	14	1468	9	1697	6	612	889	1196
580239	7	1894	6	723	9	362	362	2274
46755	1	323	0	231	0	104	117	320
22008	1	335	2	421	1	76	3282	76
19624542	247	5255	148	6455	78	2234	3451	4038
8457731	105	3742	60	3739	36	1492	1927	3309
14013790	163	4472	96	6175	56	1904	2484	2892
10840139	124	3942	73	5499	40	1673	3101	2516
820701	8	1110	4	1255	4	458	663	909
19963405	177	4637	101	6453	62	1977	3633	2986
481268	5	827	3	1087	2	360	600	586

Figure 35: Sample Profile Run1 in Nanoseconds

21827865	267	5595	163	7795	81	2394	4413	3577
2215765	29	3591	25	4029	15	726	4021	726
6191657	80	3030	44	4331	24	1273	2464	1840
38629741	231	5130	146	7013	67	2200	3948	3386
105778	1	771	3	933	2	165	2832	165
14119518	165	4690	96	6719	53	1901	4041	2550
2682336	29	2206	19	1995	14	837	1019	2033
6967378	82	3568	51	3203	29	1132	1653	3268
225205	2	669	2	969	1	230	631	270
12900415	159	4242	84	5685	49	1844	3089	3002
16428898	191	4872	109	5717	76	2061	2997	3945
11088865	134	4315	74	6245	40	1692	3804	2202
3053072	35	2100	17	2577	12	879	1368	1611
16440124	208	4834	129	6431	54	2092	3603	3323
95646	2	802	6	843	5	148	4189	148
5642971	66	3006	38	3071	23	1192	1582	2626
2952002	35	3111	24	1771	19	848	886	3110
331430	4	1384	10	577	34	289	289	4089
46	0	15	0	3	2	2	2	2276
4178846	49	2550	32	3651	15	1053	2167	1438
1311052	16	2042	11	1195	8	565	599	2013
39030648	234	5166	125	7027	66	2219	3974	3412
3617812	46	3859	34	1933	24	948	967	3850
1379189	14	1476	8	2069	5	591	1244	824
6334872	76	3112	44	3413	24	1293	1801	2606
19406179	238	5245	146	7207	71	2250	4042	3453
10570146	134	3928	69	4479	42	1638	2375	3192
834609	8	1433	7	971	4	441	488	1401
4125406	49	2418	30	3245	15	1044	1797	1605
38051860	227	5152	136	7313	69	2176	4205	3125
1948585	20	1702	15	2397	9	715	1371	1045
341886	4	1409	5	1641	5	299	2477	299
8225348	100	3877	76	3499	38	1449	1790	3545
1997641	26	3228	21	3757	10	677	3272	679
7357890	89	4264	57	2987	44	1358	1502	4135
24233500	298	5829	208	7257	92	2521	3907	4444
13744295	160	4390	111	5895	56	1904	3220	3079
17065394	196	5084	112	5665	73	2104	2946	4242
3845008	44	2547	26	2417	16	972	1245	2286
5030191	60	3573	38	2407	25	1112	1210	3488
5321818	77	4426	46	2289	41	1111	1146	4397
14188520	174	4892	103	4753	70	1915	2455	4353
16352756	196	5102	111	7335	58	2057	4407	2751
1187586	12	2505	16	2961	12	549	3434	549
17250626	235	5150	154	5733	71	2147	2951	4349
12671960	159	4456	122	4765	59	1811	2480	3790
2723272	29	1995	18	2843	11	858	1589	1264

Figure 36: Sample Profile Run2 in nanoseconds.

21760	1	379	1	421	1	72	1288	72
420445	7	1551	10	653	21	327	327	3606
4643617	61	3353	35	2341	24	1064	1179	3241
4224287	54	3847	41	4899	17	983	3807	1022
3420554	36	2628	28	3735	12	915	2480	1062
17327673	224	4977	125	6763	62	2137	3802	3314

Figure 37: Sample Profile Run3 in nanoseconds.

The three images in figure 38, 39 and 40 are a demonstration of the profile run of each one of the algorithms in microseconds.

N-R-C	N-CT (ms)	M-RC	M-CT (ms)	D-RC	D-CT (ms)	Q-RC	Q-CT (ms)	LCS	ISL1	ISL2
1	0	1	0	1	1	0	0	0	0	18
98	0	44	0	6	0	5	0	2	3	11
432	0	61	0	12	0	9	0	1	11	3
8	0	8	0	5	0	1	0	1	1	14
180205	22	237	0	22	0	25	0	9	13	18
1	0	1	0	1	0	1	0	0	0	21
8E+06	39	348	0	26	0	25	0	11	13	23
7	0	7	0	5	0	5	0	2	2	5
19595	0	146	0	17	1	13	0	5	8	14
72569	0	218	0	25	0	13	0	6	8	23
-2E+08	12489	614	0	31	0	39	1	10	23	19
18	0	18	0	10	0	1	0	1	1	10
159	0	39	0	8	0	7	0	3	4	7
53	0	23	0	8	0	17	0	2	23	2
20	0	19	0	9	0	13	0	2	16	2
153972	1	191	0	17	0	23	0	6	14	10
563	0	72	0	13	0	9	0	4	5	11
31	0	20	0	10	0	15	0	3	20	3
6	0	6	0	3	0	3	0	1	2	3
1957	0	128	0	22	0	31	0	3	23	4
31594	0	161	0	17	0	13	0	2	12	7
2	0	2	0	2	0	11	0	1	20	1
2E+08	748	565	0	27	0	37	0	12	21	20
1E+07	36	398	0	23	0	29	0	8	15	17
695327	2	260	0	18	0	27	1	8	16	12
3159	0	97	0	14	0	11	0	3	12	6
3745	0	84	0	15	0	25	0	7	18	7
10	0	10	0	6	0	9	0	1	13	1
5575	0	104	0	14	0	17	0	7	9	14
2E+09	5282	615	0	32	0	37	0	12	21	22
19596	0	141	0	11	1	15	0	6	8	14
48332	0	186	0	24	0	21	0	6	23	8
1	0	1	0	1	0	1	0	0	16	0
9E+08	16225	880	1	34	0	39	0	12	22	24
2E+07	65	403	0	26	0	21	0	8	13	23
1E+07	48	379	0	22	0	23	1	5	16	15
2E+08	565	519	0	25	1	27	0	10	17	21
371893	1	187	0	19	0	29	0	8	18	12
46026	0	167	0	14	0	15	1	4	9	11
-2E+09	91246	740	0	32	0	37	0	10	20	24
14445	0	183	0	16	0	17	0	7	9	14
79839	0	228	0	19	0	27	0	8	14	13
103	0	47	0	9	0	5	0	2	3	11
1	0	1	0	1	0	1	0	0	21	0
7161	0	126	0	24	0	25	1	5	23	6
3E+07	94	440	0	26	0	29	1	8	22	13

Figure 38: Sample Profile Run1 in milliseconds.

566	0	59	0	12	0	9	0	2	10	4
92	0	29	0	5	0	7	0	1	4	4
2	0	2	0	2	0	11	0	1	24	1
4946	0	93	0	10	0	11	0	5	10	9
204	0	44	0	10	0	5	0	2	3	9
1	0	1	0	1	0	1	0	0	0	24
503	0	51	0	12	0	9	0	5	5	11

Figure 40: Sample Profile Run3 in milliseconds.

133794	0	211	0	21	0	31	1	8	20	9
1	0	1	0	1	0	1	0	0	3	0
2E+09	4627	557	0	28	0	35	0	10	21	20
3809	0	141	0	25	0	31	1	5	24	6
3154	0	93	0	13	0	11	0	3	10	6
418969	2	249	0	23	0	15	1	5	9	21
118556	0	205	0	18	1	21	0	8	11	14
5910	0	86	0	15	0	19	0	4	13	5
577	0	70	0	11	0	15	0	3	8	5
4E+06	20	353	0	28	0	39	0	9	24	13
10	0	10	0	5	0	11	1	2	10	2
8	0	8	0	5	0	9	0	1	16	1
13	0	13	0	7	0	7	0	0	6	1
2E+09	18950	744	0	30	0	43	1	13	24	22
273	0	56	0	9	0	13	0	3	7	5
765	0	64	0	10	0	9	0	3	5	8
3649	0	131	0	21	0	25	0	8	20	9
88	0	44	1	15	0	21	0	4	20	4
61	0	29	0	12	0	5	0	3	3	23
124	0	48	0	14	0	25	0	3	24	3
1E+08	354	520	0	28	0	31	1	10	21	17
14	0	14	0	8	0	5	0	1	7	1
33861	0	169	0	19	0	21	0	4	17	6
2E+07	62	458	0	25	0	35	0	9	19	15
1E+07	33	413	0	23	0	25	0	8	15	18
2E+06	7	371	0	22	0	41	0	10	22	13
774810	2	248	0	17	0	21	0	5	12	13
526	0	61	0	20	0	7	0	4	4	22
8	0	8	0	5	0	5	0	1	5	1
18	0	15	0	5	0	5	0	3	3	12
-3E+08	11500	573	0	31	0	35	0	11	22	22
220423	1	258	0	18	0	21	0	7	14	14
3E+08	822	621	0	25	0	35	0	11	18	23
184	0	64	0	22	0	15	0	2	21	2
7E+06	20	347	0	23	0	23	1	8	13	19
245017	0	254	0	17	0	25	0	6	16	10
2E+07	52	369	0	23	0	29	0	10	16	19
799	0	67	0	8	0	11	0	1	7	5
32819	0	156	0	11	0	15	0	4	13	7
28902	0	171	0	16	0	17	1	6	9	12
78	0	43	0	9	0	3	0	1	2	15
1	0	1	0	1	0	1	0	0	0	11
301854	1	252	0	13	0	17	0	7	10	20
5913	0	109	0	14	0	17	0	4	11	7
10	0	9	0	4	0	3	0	2	2	6
244	0	57	0	11	0	9	0	4	5	10
7E+07	200	516	0	27	0	29	1	10	16	24

Figure 39: Sample Profile Run2 in milliseconds.

4. CONCLUSION

This experimental project served to us as a great experience to learn more about the implementation of different algorithms such as Naïve Recursive, Memoization version, Dynamic programming, and Hirschberg. Each one of these algorithms implementation was based on the LCS computation, which was generated from two input strings. These input strings, in our case, were in two forms. First, we tested the algorithms with random whose length was varying randomly. Second, we tested the algorithms with strings of equal and different length, which were increasing linearly. Lastly, all tests were recorded by high level scripts in java, and analyzed by high level scripts in Matlab. Furthermore, we were able to learn how to utilize certain tools which were crucially important for the development of this report. These tools are Matlab and L^AT_EX.

Matlab, we learned how to process and analyze data by classification. Using the necessary data for performance analysis, which includes the time efficiency and number of recursive calls.

L^AT_EX, we learned how to utilize this tool to professionally write our report. Also, we were able to learn how to manage image and formatting to technically write our documentation.

In fact, the implementation of this project helped us to understand how to analyze the behavior of an algorithm. The factors that depend on the complexity of such algorithm. In brief, it taught us the importance of the performance of any algorithm. This means, the complexity of the algorithm matters as much as the memory usage for the development process. This would affect any system even if we have modern computer with advanced GPU.